

第六章：语义分析

语义检查概述
标识符的语义表示
类型的语义表示

1.1 语法和语义的区别

- ◆ 语言包括4个内容：词法、语法、语义、语用
- ◆ 词法：关于程序中允许出现哪些字符，有哪些单词，每类单词如何构成的规则。
- ◆ 语法：关于什么样的字符串才是该语言在组成结构上合法的程序的法则。
- ◆ 语义：关于结构上合法的程序的意义法则。

1.2 语义分析的必要性

- ◆ 一个语法正确的程序不能保证它是有意义的.
- ◆ 程序中容易出现各种语义错误：
 - 标识符未声明
 - 标识符重复声明
 - 操作数的类型与操作符的类型不匹配
 -

1. `int x=10;` _____ 符合变量声明的语法、语义

2. `int main( )`

3. `{ printf( "%d", x+x );`

4. `x( );` _____ 符合函数调用的语法、不符合语义

5. `f = x;` _____ 符合赋值语句的语法、不符合语义

6. `}`

7. `float f( )` _____ 符合函数声明的语法、语义

8. `{int x=20, y;` _____ 符合变量声明的语法、语义

9. `float x;` _____ 符合变量声明的语法、不符合语义

10. `printf( "%d", x );`

11. `}`

1.3 程序设计语言语义的分类

◆ 静态语义

- 在编译阶段 (compile-time) 可以检查的语义
- 例如：标识符未声明、标识符重复声明、变量标识符当作函数标识符使用、……

◆ 动态语义

- 目标程序运行时 (run-time) 才能检查的语义
- 例如：除零溢出错误、数组下标越界错误、……

注：编译程序只能进行静态语义检查

1.4 语义分析的主要任务

◆ 1. 根据声明部分建立符号表

符号表 (symbol table)：是一种供编译器用于保存有关源程序的各种信息的**数据结构**。符号表的每个条目中包含与一个**标识符**相关的信息，这些信息全面地反映该名字的属性及它们在编译过程中的特征。

符号表的作用：

- 存储标识符的属性；
- 便于检查语义错误；
- 代码生成阶段作为地址分配的依据。

1.4 语义分析的主要任务

◆ 2. 在整个程序范围内检查常见语义错误

- 标识符声明和使用相关的错误
- 类型相关的语义错误

1.5 语义错误检查

◆ 标识符声明和使用相关的语义错误

● 常见的语义错误：

- 每个使用性标识符是否都有声明？在同层内有无标识符被声明多次？
- 标号是否有声明？有无重复声明和重复定位错误？有无非法转入错误？

● 如何检查？

■ 每当遇到新声明的标识符，查符号表：

- 如果当前有效的所有标识符中有相同名字，则是重复声明错误；
- 否则生成它的属性信息，保存到符号表中；

■ 每当遇到标识符的使用，查符号表

- 如果没有找到，说明该标识符没有声明；
- 如果找到，则可获取该标识符的属性，进行进一步分析；⁸

1.5 语义错误检查

◆ 类型相关的语义错误

- 各种条件表达式的类型是不是boolean型?
- 运算符的分量的类型是否相容?
- 赋值语句的左右部的类型是否相容?
- 形参和实参的类型是否相容?
- 下标表达式的类型是否为所允许的类型?

1.5 语义错误检查

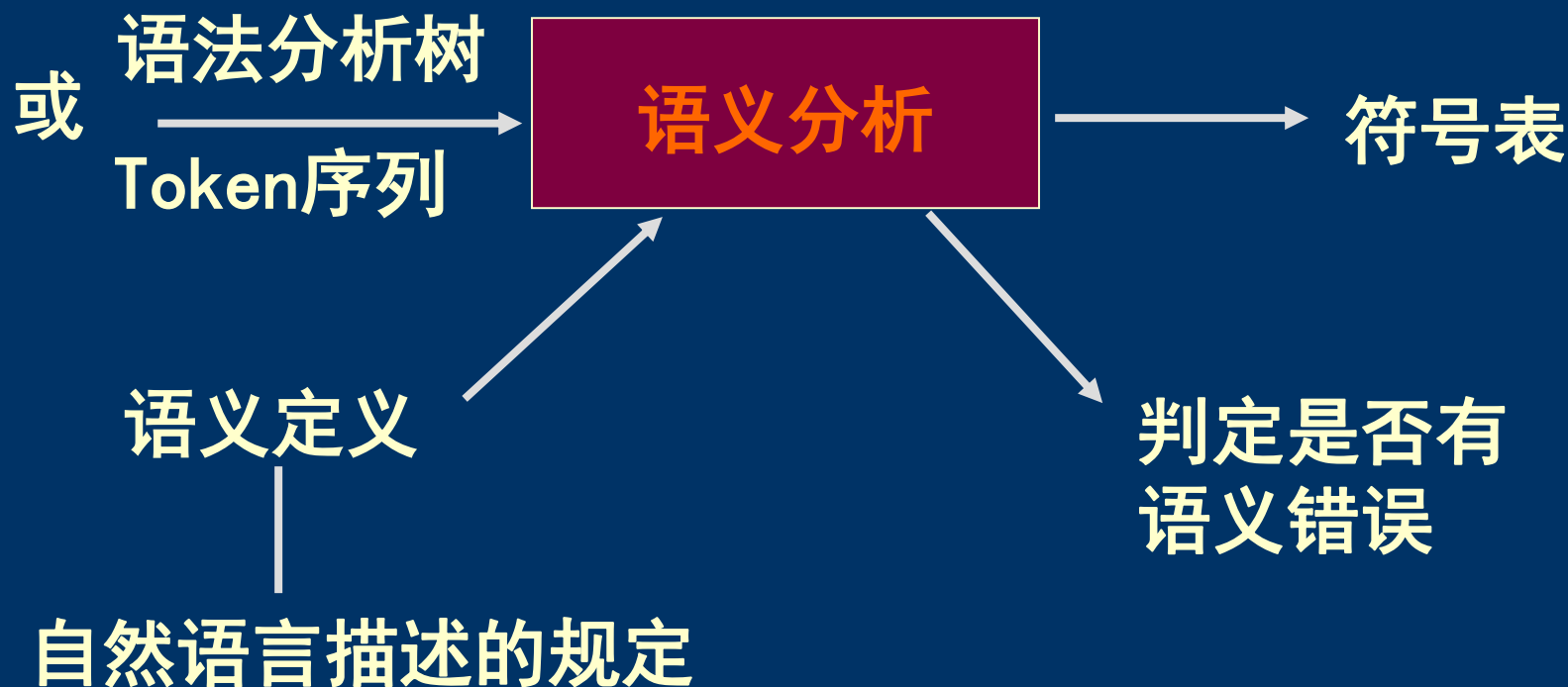
- 函数说明中的函数类型和返回值的类型是否一致?
- $V[E]$ 中的 V 是不是变量, 而且是不是数组类型?
- $V.id$ 中的 V 是不是变量, 而且是不是结构体类型? id 是不是该结构体类型中的成员?
- $V \uparrow (*V)$ 中的 V 是不是指针或文件变量?
- $y+f(\dots)$ 中的 f 是不是函数名? 形参个数和实参个数是否一致?

1.6 语义分析的实现

方式一：不作为独立的一遍

- ① 类型语义错误检查：可以安排在中间代码生成时进行
- ② 一般的语义检查：与语法分析相结合

方式二 独立一遍的语义分析的功能图示



2.1 标识符在程序中的出现

- ◆ 声明性出现

如： `int x, int a[10];`

Pascal语言中出现在声明部分、函数头部分

- ◆ 使用性出现

如： `x=x+1; a[0]=a[1]+a[2];`

Pascal语言中出现在程序体、函数体部分

2.2 标识符的种类

- ◆ 常量标识符
- ◆ 类型标识符
- ◆ 变量标识符
 - 实在变量
 - 形参变量
 - 值引用型 地址引用型
- ◆ 过函标识符
 - 实在过函
 - 形式过函
- ◆ 域名标识符

2.3 标识符的属性

- ◆ 标识符的主要信息主要包括以下几个：
 1. 名字
 2. 种类信息(kind:常量、变量、函数、类型等)
 3. 类型信息(type:整型、实型、数组、结构体、指针、枚举等)
 4. 针对不同种类标识符的特有的信息（数组：下届、上界、元素类型；函数：形式参数的类型、参数传递的方式等）

2.4 标识符的语义表示

◆ 常量标识符

Name	Kind	Type	Value
------	------	------	-------

其中各个域的含义如下：

- ❖ **Name** 是常量的名字；
- ❖ **Kind** = `constKind`，表明该标识符是常量标识符；
- ❖ **Type** = `TypePtr`，其中 `TypePtr` 是指向具体常量的类型的内部表示的指针；
- ❖ **Value** = `ValPtr`，其中 `ValPtr` 是指向具体常量值的内部表示的指针。

例： C语言的“常量定义”：

```
#define pai 3.14
```

```
#define count 100
```

常量标识符pai 和count的内部表示为：

pai	constKind	realPtr	↑<3.14>
count	constKind	intPtr	↑<100>

2.4 标识符的语义表示

- ◆ 类型标识符

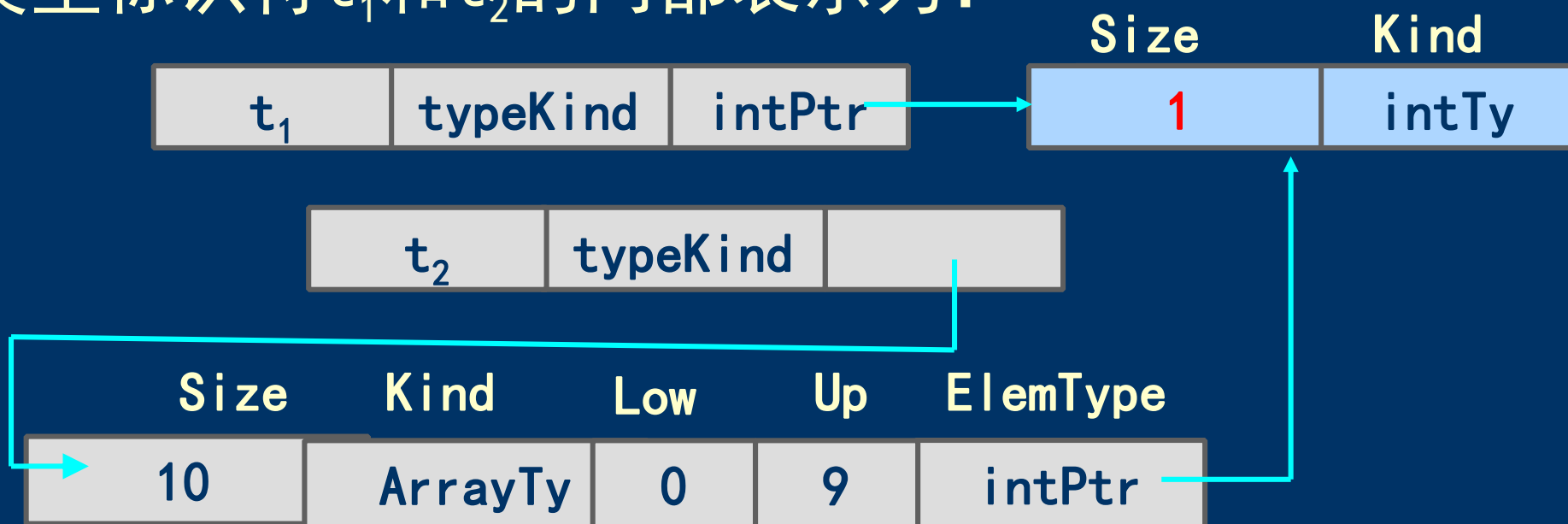
Name	Kind	Type
------	------	------

- ❖ **Name**是类型标识符的名字；
- ❖ **Kind** = typeKind, 表示标识符是类型标识符；
- ❖ **Type** = TypePtr, 指向类型标识符指代的类型的内部表示。

例：
C语言的类型定义：

```
typedef    int    t1;  
typedef    int    t2 [10];
```

类型标识符 t_1 和 t_2 的内部表示为：



2.4 标识符的语义表示

◆ 变量标识符的内部表示

Name	Kind	Type	Access	Level	Off	Value
------	------	------	--------	-------	-----	-------

- ❖ **Kind** = varKind, 表明该标识符是变量标识符;
- ❖ **Access** = dir表示变量是直接变量, **Access** = indir表示变量是间接变量;
- ❖ **Level**表示该变量声明所在主程序/函数/过程的层数;
- ❖ **Off**表示该变量相对它所在主程序/函数/过程的内存块起始地址的偏移量;
- ❖ **Value** = ValPtr, 如果变量定义时说明了初值, 则为初值的内部表示的指针, 否则为空。

例： C语言的变量声明：

```
int      x=10;  
float    y;  
float*   z;
```

变量标识符x、y和z的内部表示为（当前层为L，当前偏移量为off）：

x	varKind	intPtr	dir	L	off	↑<10>
y	varKind	realPtr	dir	L	off+1	NULL
z	varKind		indir	L	off+3	NULL



1	pointTy	realPtr
---	---------	---------

2.4 标识符的语义表示

◆ 过程/函数标识符的内部表示

Name	Type	Kind	Level	off	Param	Class	Code	Size	Forward
------	------	------	-------	-----	-------	-------	------	------	---------

- ❖ **Type**表示函数返回值类型的内部表示(过程情形是NULL)
- ❖ **Kind** = routKind;
- ❖ **Level**表示过/函的层数;
- ❖ **Off**只对形式过/函有效, 表示形式过/函在所属过/函内存块中的偏移;

2.4 标识符的语义表示

◆ 过程/函数标识符的内部表示

Name	Type	Kind	Level	off	Param	Class	Code	Size	Forward
------	------	------	-------	-----	-------	-------	------	------	---------

- ❖ **Param**表示过/函的参数表指针，参数表的结构同符号表的结构相同，参数信息可以填入符号表，也可以填入单独的参数表当中（对于删除式符号表，则既需要填入符号表，也需要填入单独的参数表）；
- ❖ **Class**= actual表示实在过/函，Class = formal表示形式过/函；
- ❖ **Code**只对实在过/函有效，表示过/函定义对应生成的目标代码的起始地址，当目标代码生成时回填得到，形式过/函的code为NULL；

2.4 标识符的语义表示

◆ 过程/函数标识符的内部表示

Name	Type	Kind	Level	off	Param	Class	Code	Size	Forward
------	------	------	-------	-----	-------	-------	------	------	---------

- ❖ **Size**只对实在过/函有效，表示过/函的过程活动记录所占内存区的大小，也要当目标代码生成以后回填得到；
- ❖ **Forward** 属性只对实在过/函有效，Forward= true表示过/函是超前声明，Forward = false表示过/函不是超前声明。（过/函的超前声明对应C语言中的函数原型）

```
int  add(int x, int y); //函数声明或函数原型
```

```
....
```

```
int  add(int x, int y){ return x+y;} //函数定义
```


例：Pascal语言的函数标识符定义：

function f(x: integer; var y: real;

function inc(a: integer): integer): integer; "头"

begin

.....f的函数体部分

end;

Name	TypePtr	Kind	Level	off	Parm	Class	Code	Size	Forward
------	---------	------	-------	-----	------	-------	------	------	---------

f	intPtr	routKind	L			actual			false
---	--------	----------	---	--	--	--------	--	--	-------

x	varKind	intPtr	dir	L+1	off ₀
---	---------	--------	-----	-----	------------------

y	varKind		indir	L+1	off ₀ +1
---	---------	--	-------	-----	---------------------

inc	intPtr	routKind	L+1	off ₀ + 2		formal			
-----	--------	----------	-----	-------------------------	--	--------	--	--	--

1	pointTy	realPtr
---	---------	---------

a	varKind	intPtr	dir		
---	---------	--------	-----	--	--

3. 类型的语义表示

- ◆ 类型语义信息的作用
- ❖ 类型的语义检查
- ❖ 分配空间的大小

3. 类型的语义表示

- ◆ 类型可以分成下面几大类：
 - ❖ 标准的类型：整型、实型、bool、字符类型，这是标准的数据类型
 - ❖ 自定义的数据类型：子界类型，枚举类型
 - ❖ 结构数据类型：数组，记录（结构），共用体
 - ❖ 指针类型
- ◆ 类型存储占用空间大小size属性：

为方便起见，规定RealSize取2, IntSize、BoolSize、CharSize取1。

3. 类型的语义表示

◆ 标准类型： 大小、种类

	Size	Kind
intPtr→	IntSize	intTy
boolPtr→	BoolSize	boolTy
charPtr→	CharSize	charTy
realPtr→	RealSize	realTy

Size	Kind
1	intTy
1	boolTy
1	charTy
2	realTy

3. 类型的语义表示

◆ 数组类型

Size	Kind	Low	Up	ElemType
ArraySize	ArrayTy			TypePtr

- ❖ **Size**: $\text{Size} = (\text{Up} - \text{Low} + 1) * \text{sizeof}(\text{ElemType})$
- ❖ **Kind** = ArrayTy, 表示是数组类型;
- ❖ **Low**表示数组下标的下界, 在C语言中Low = 0;
- ❖ **Up**表示数组下标的上界;
- ❖ **ElemType** 表示数组成分类型的内部表示指针。

例：C语言的数组

```
typedef int A[10];  
typedef char B [5] [10];
```

A和B的内部表示分别为：



3. 类型的语义表示

◆ 结构体和联合体

Size	Kind	RecBody
------	------	---------

- ❖ **Kind**=structTy 表示结构体类型
- ❖ **Kind**=unionTy表示联合体类型

◆ RecBody的内部表示如下

Name	Type	Off	link
------	------	-----	------

Name表示域名

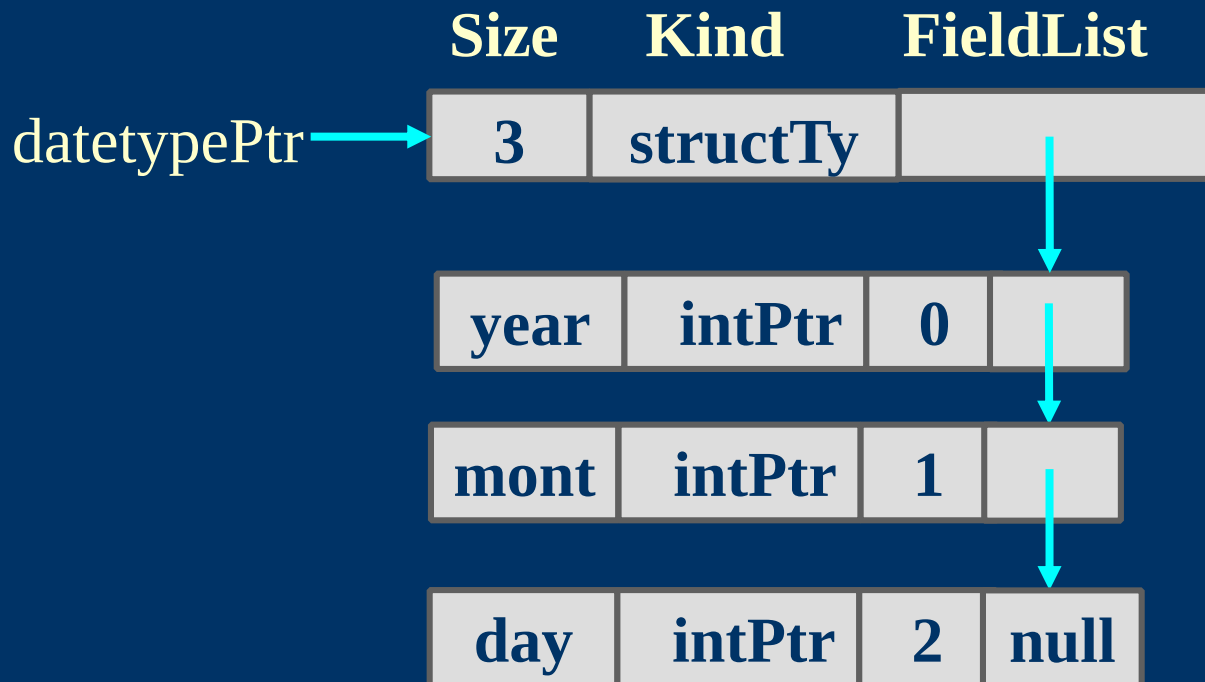
Type指向域的类型

Off表示纪录域相对于结构体类型分配的内存块起始地址的偏移量，对于联合类型而言，所有的域名标识符的起始偏移都是相同的，所以可以省略；

link指向下一个域名标识符内部表示的指针

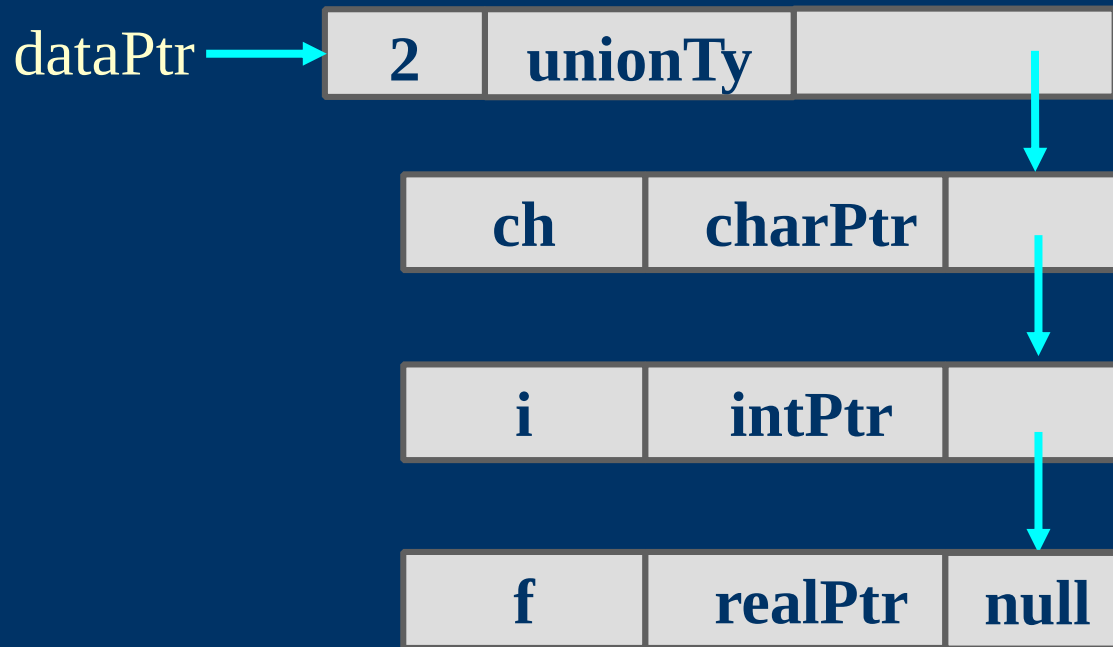
例:

```
typedef struct Date  
{int year, month, day;} datatype;
```



例：c语言的联合体

```
typedef union {char ch; int i; float f;} data;
```



3. 类型的语义表示

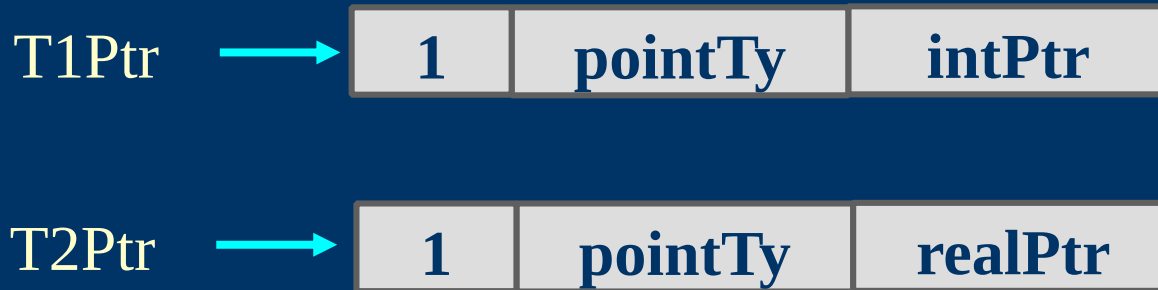
◆ 指针类型

Size	Kind	BaseType
------	------	----------

- ❖ **size**表示指针类型所占空间的大小，指地址的长度（一般一个单元）
- ❖ **Kind** = `pointTy`表示是指针类型；
- ❖ **BaseType**表示指针所指向空间的类型

例：c语言的针举类型

```
typedef int* T1;  
typedef float* T2;
```



3. 类型的语义表示

- ◆ 枚举类型

Size	Kind	ElemList
------	------	----------

- ❖ **size**表示枚举类型所占空间的大小;
- ❖ **Kind** = enumTy;
- ❖ **ElemList**是指向枚举常量表表头的指针.

- ◆ 枚举常量表内部表示:

Name	Value
------	-------

- ❖ **Name**表示枚举常量的名字;
- ❖ **Value**表示枚举常量所代表的整数值。

例：c语言的枚举类型
enum color {red, yellow, blue}

1	enumTy	
---	--------	--



red	0
yellow	1
blue	2

例：c语言的枚举类型
enum color {red=10, yellow=red+2, blue}

1	enumTy	
---	--------	--



red	10
yellow	12
blue	13